ADVANCED DYNAMIC PROGRAMMING SERIES

Subset & Partition DP Transformations

Mathematical reduction of Target Sum to Count Subset Sum DP

This is one of the most important DP transformations in the entire Subset/Partition DP family. Once you understand this conversion, you'll be able to solve:

- Target Sum (Leetcode 494)
- Count Partitions with Given Difference
- Partition With Given Difference
- Count Subsets With Sum K
- Many interview variations

1. Original Problem

Given:

```
nums = [1,1,1,1,1]
target = 3
```

We need to place `+` or `-` before every element.

Example:

```
+1 +1 +1 +1 -1 = 3
```

Goal is to count how many such expressions evaluate to target.

2. What are we actually doing?

Suppose:

```
nums = [a,b,c,d,e]
```

Some numbers get a `+` and some numbers get a `-`. For example:

```
+a -b -c +d +e
```

Let's separate them into two groups.

Positive group:

```
S1 = a + d + e
```

Negative group:

```
S2 = b + c
```

Notice that the overall Expression can be written as:

```
Expression = S1 - S2
```

because S2 numbers are assigned minus signs.

3. First Equation

The final expression must equal target. Therefore:

> $S_1 - S_2 = target$

This is our first equation.

4. Second Equation

Every element belongs to either S1 or S2. Therefore:

> $S_1 + S_2 = totalSum$

Example:

```
nums = [1,1,1,1,1]
totalSum = 5
```

Then:

```
S1 + S2 = 5
```

This is our second equation.

5. Solve the Equations

We have:

> $S_1 - S_2 = target$
> $S_1 + S_2 = totalSum$

Add both equations:

> $2S_1 = totalSum + target$

> $S_1 = (totalSum + target) / 2$

We can also solve for S2 by subtracting the equations:

> $2S_2 = totalSum - target$

> $S_2 = (totalSum - target) / 2$

6. The Magic Transformation

Instead of placing '+' and '-' signs, we only need to find:

> How many subsets have sum = (totalSum - target) / 2
> or equivalently
> How many subsets have sum = (totalSum + target) / 2

Most implementations use:

```
(targetSubsetSum) = (totalSum - target) / 2
```

because it matches the Partition Difference problem.

7. Why does this become Partition Difference?

Recall the Partition Difference problem: We divide an array into two subsets `S1` and `S2` such that:

> $$S_1 - S_2 = D$$

and count ways. Target Sum also creates two groups:

- Positive numbers → `S1`
- Negative numbers → `S2`

and satisfies:

> $$S_1 - S_2 = target$$

This is literally the same equation. Therefore:

```
Target Sum = Count Partitions With Difference = target
```

They are the same DP problem after transformation.

8. Example Dry Run

Array configuration:

```
nums = [1,1,1,1,1]
target = 3
```

Total sum evaluates to `5`. Compute S2:

> $$S_2 = (5 - 3) / 2$$
> $$S_2 = 2 / 2 = 1$$

Now the question becomes:

> Count subsets whose sum = 1

Possible subsets:

```
[1], [1], [1], [1], [1]
```

There are exactly 5 ways. Answer: `5`. This matches the original Target Sum answer completely.

9. Edge Case 1

Suppose:

```
nums = [1,2,3]
target = 10
```

Total sum is `6`. The maximum possible expression is `+1 +2 +3 = 6`. We can never reach 10. Therefore:

```
if(totalSum < target)
    return 0;
```

10. Edge Case 2

Suppose:

```
totalSum = 7
target = 2
```

Then computing S2 gives:

> $S_2 = (7 - 2) / 2 = 5 / 2$

This is not an integer. It is impossible because subset sums must be integers. Therefore:

```
if((totalSum - target) % 2)
    return 0;
```

11. Why does Count Subset Sum DP solve it?

After conversion:

targetSumWays → count subsets having sum = S_2

Now this becomes the classic problem: **Count Subsets With Sum K** where:

```
K = (totalSum - target) / 2
```

That's why we directly reuse the DP algorithm.

12. DP State

Define:

```
f(ind, target)
```

Meaning: Number of subsets from index `0...ind` whose sum equals target.

13. Choices

At every index, we have two choices:

Choice 1: Not Pick

```
f(ind-1, target)
```

Ignore the current element.

Choice 2: Pick

```
f(ind-1, target - arr[ind])
```

Include the current element. This choice is only allowed when:

```
arr[ind] <= target
```

14. Recurrence

```
pick = f(ind-1, target - arr[ind]);
notPick = f(ind-1, target);

return pick + notPick;
```

We use addition because we are counting total valid ways.

15. Most Important Base Case (with Zero Handling)

For counting subset sums accurately when array includes zeros:

```
if(ind == 0)
{
    if(target == 0 && nums[0] == 0)
        return 2;

    if(target == 0 || target == nums[0])
        return 1;

    return 0;
}
```

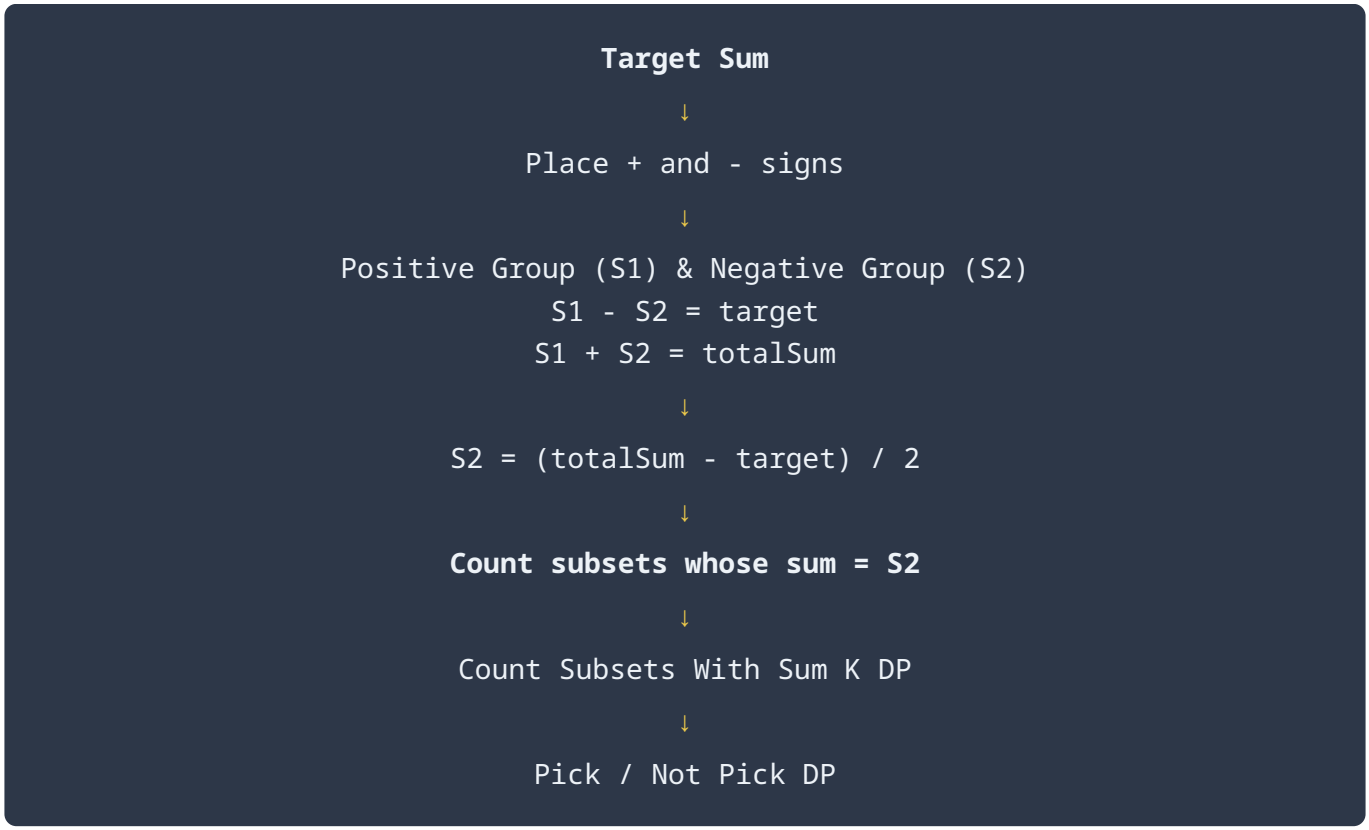
Why return 2?

For `nums[0] = 0` and `target = 0`, two valid subsets exist:

```
{}  and  {0}
```

Both subsets produce a sum of 0. Hence, we must return 2.

Complete Mental Model



This reveals the core structural identity:

Target Sum = Count Partitions With Given Difference = Count Subsets With Sum (totalSum - target) / 2

Once you recognize the equations $S_1 - S_2 = target$ and $S_1 + S_2 = totalSum$, the entire problem immediately reduces to a standard Count Subset Sum DP execution pattern.